Installing Open MPI on your machine

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Lab 4 needs an MPI library and its launcher on your own machine: Compiler Explorer runs a single program and cannot launch several processes. We use **Open MPI**, which provides the compiler wrapper `mpic++` and the launcher `mpirun`. This guide covers the three cases: Ubuntu Linux, Windows (through WSL, the Windows Subsystem for Linux, which gives you an Ubuntu inside Windows), and macOS (through Homebrew). In every case, the commands are typed in a terminal; the installation takes a few minutes and a network connection. At the end, section 4 tells you how to check that everything works.

Part 1

Ubuntu Linux

This applies to Ubuntu (22.04 or 24.04) and to the other Debian-based distributions (Debian, Linux Mint, ...), including the Ubuntu of WSL on Windows (section 2). Open a terminal and install the C++ compiler and Open MPI with the package manager:

```
sudo apt update && sudo apt install -y build-essential openmpi-bin libopenmpi-dev
```

`sudo` asks for your password (nothing is displayed while you type it). `build-essential` provides `g++` and `make`; `openmpi-bin` provides `mpirun`; `libopenmpi-dev` provides `mpic++` and the header `mpi.h`. That is all; go to section 4.

On another Linux distribution, install the equivalent packages with its package manager: on Fedora, `sudo dnf install openmpi openmpi-devel` then `module load mpi/openmpi-x86_64` in every new terminal; on Arch Linux, `sudo pacman -S openmpi`.

Part 2

Windows: Ubuntu through WSL

MPI programs are not developed on Windows directly: we install *WSL*, which runs a real Ubuntu inside Windows (Windows 10 version 2004 or later, or Windows 11), then install Open MPI in that Ubuntu exactly as in section 1.

1. Open *PowerShell* or the *Terminal* **as administrator** (right-click on it in the Start menu, "Run as administrator") and type

   ```
   wsl -install
   ```

   This installs WSL together with the Ubuntu distribution. When it is done, **restart the computer**.

2. After the restart, Ubuntu opens by itself (if not, open "Ubuntu" from the Start menu) and finishes its installation, which takes a minute. It asks you to choose a *Unix user name* and a *password*: this is the account you will use inside Ubuntu, independent of your Windows account; nothing is displayed while you type the password. You then get an Ubuntu terminal (a prompt ending with `$`).

3. In this Ubuntu terminal, install Open MPI as in section 1:

   ```
   sudo apt update && sudo apt install -y build-essential openmpi-bin libopenmpi-dev
   ```

From now on, everything of the lab happens in the Ubuntu terminal ("Ubuntu" in the Start menu, or the *Ubuntu* tab of the Windows Terminal), never in PowerShell or `cmd`. A few things to know:

- Your Ubuntu has its own home folder, `/home/<user>` (`~`), which is the place to keep the lab files: create a folder there, `mkdir ~/lab4`, and work in it, `cd ~/lab4`.

- Your Windows files are visible from Ubuntu under `/mnt/c/`: for instance `/mnt/c/Users/<name>/Downloads/` is your Windows *Downloads* folder; you can copy a file from there with `cp`. Conversely, in the Windows file explorer, the Ubuntu files are under `Linux > Ubuntu > home > <user>` in the left panel, or at the address `\\wsl$\Ubuntu\home\<user>`.
- To edit the files with VS Code: install the *WSL* extension in VS Code, then type `code .` in the Ubuntu terminal from the lab folder; VS Code opens on that folder and its integrated terminal is the Ubuntu one.
- If `wsl -install` fails saying that virtualization is disabled, enable it in the BIOS/UEFI settings of the computer ("Intel VT-x", "AMD-V" or "SVM"); if it reports that WSL is already installed, install Ubuntu alone with `wsl -install -d Ubuntu`.

Part 3

macOS: Homebrew

Open the *Terminal* application (Applications > Utilities, or search "Terminal" with Spotlight).

1. If you do not have *Homebrew*, the package manager of macOS, install it by pasting the command given on `https://brew.sh`, which is currently

   ```
   /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
   ```

   It asks for your password, installs the *Command Line Tools* of Xcode if they are missing (the C++ compiler, among others; this part is long), then prints two or three lines to run so that the `brew` command is found in your terminal ("Next steps"): run them. Open a new terminal and check that `brew -version` answers.

2. Install Open MPI:

   ```
   brew install open-mpi
   ```

That is all; go to section 4. On macOS, `mpic++` calls Apple's C++ compiler (`clang++`), which is fine for this lab.

Part 4

Checking the installation

Open a new terminal and check that the two commands are found:

```
mpic++ -version     mpirun -version
```

Then compile and run a first program: save the following lines in a file `hello.cpp` (or use the `hello.cpp` of the lab once you have written it), in a folder of your own, and in that folder:

```
mpic++ -O2 hello.cpp -o hello     then     mpirun -np 4 ./hello
```

```
#include <cstdio>
#include <mpi.h>

int main(int argc, char **argv)
{
  MPI_Init(&argc, &argv);
  int rank, size;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  printf("I am process %d of %d\n", rank, size);
  MPI_Finalize();
  return 0;
}
```

You must get four lines `I am process k of 4`, $k = 0, \ldots, 3$, in any order. The number after `-np` is the number of processes; `nproc` (Linux) or `sysctl -n hw.ncpu` (macOS) tells you how many cores your machine has.

Part 5

If something goes wrong

- `mpic++: command not found` or `mpirun: command not found`: the installation did not complete, or the terminal was opened before it; open a new terminal and retry, then redo the installation command of your system and read its messages. On macOS, make sure that the "Next steps" lines printed by the Homebrew installer were run.
- `There are not enough slots available in the system`: you asked for more processes than your machine has cores; add `-oversubscribe` after `mpirun`:

```
mpirun -oversubscribe -np 8 ./hello
```

- macOS asks whether the program may *accept incoming network connections*: answer *Allow*; the processes talk to each other through the network layer of the machine.
- On Windows, the commands must be typed in the Ubuntu terminal (section 2), not in PowerShell or `cmd`; and the files must be in a folder that Ubuntu sees (its home, or `/mnt/c/...`).
- `mpirun` refuses to run as root ("root" user): work with your normal user account.